

经修改后起到攻击效率提升的代码结构如下：

1. 原本模型结构没有被优化。
2. 旧版 model.py 中数据标准化代码如下：

```
# 使用了 Normalize，数据范围不再是 [0,1]
transform=transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,)), # 标准化到均值0.1307，标准差
0.3081
])
```

FGSM 的 $\text{eps}=0.05$ 是在标准化空间中的步长，对应原始像素空间的步长约为 $0.05 * 0.3081 \approx 0.015$ ，导致扰动变化过小，修改后如下：

```
# 只做 ToTensor，数据范围保持在 [0,1]
transform = transforms.ToTensor()
```

直接作用于 $[0, 1]$ 的像素，更大的扰动幅度让攻击率变高。

3. 此前提到将单步测试优化成多步。
4. 采用前景掩码，将有限的扰动预算集中在关键的前景区域，相当于在有效区域增加了扰动强度。

```
def foreground_mask(data, threshold=0.3, dilation=3):
    mask = (data > threshold).float()
    if dilation > 1:
        mask = F.max_pool2d(mask, kernel_size=dilation, stride=1,
padding=dilation // 2)
    return mask

# 攻击时只扰动前景区域
adv_batch = fgsm_step(model, batch, batch_labels, eps,
perturb_mask=batch_mask)
```